Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Integrating stacks and queues

Today
Welcome to your next lesson: Integrating stacks and queues.

I'm getting your personalized lesson ready — based on what you've learned, what's ahead, and how you learn best.

Step by step, you're moving forward.

Ron, let's plug your queue knowledge into something new: how queues and stacks behave differently 🎯

Quick picture:

Stack (LIFO) = plate stack in a cafeteria → last plate put on top is the first one you take off.
Queue (FIFO) = grocery checkout line → first person in line is the first one served.

So the "direction" of time is opposite:

Structure	Order rule	Who gets processed next?
Stack	LIFO	The newest item
Queue	FIFO	The oldest item

We'll use both together soon (people in a queue each holding a stack of items), but first:

👉 In a "browser back button" feature, where you go back to the most recent page you visited, which fits better: a stack or a queue?

stack

Exactly — stack is the right choice ☑

Reason (step by step):

When you visit a new page, it goes on top of the history.
When you press "Back", you always want the most recent page first.
That's pure LIFO → last visited, first to come back.
Now let's flip it 🔁

Imagine a print queue in an office: documents arrive and wait to be printed.

The one that arrived first should be printed first (to be fair).

For this printing system, which fits better: stack or queue?

queue

Yes, perfect — queue for the printer 🖨️☑️

First document in → first document out → FIFO.

Now I want to show how both can work together in one little adventure.

Picture this: there is a line of adventurers waiting to enter a dungeon (that's a queue).

Each adventurer is wearing a backpack where they toss items on top and take from the top (that's a stack).

Very simple mental model:

The line of people → queue
Each person's backpack → stack

Does that picture make sense so far, or should I draw a small visual for it next? 🎒🧏‍♂️

The picture makes sense

Nice 👍 that's all we need to start using both.

Let's make it a bit more concrete with steps.

We'll model:

queue = line of adventurers (in order)
each adventurer = a name plus a stack of items (their backpack)

In Python, we can store this as a list of lists (or tuples), like:

outer list = the queue
inner list = one adventurer's stack of items

Very rough idea:

adventure_queue = [...] ← people in line
each person: ["Alice", ["sword", "potion"]] where ["sword", "potion"] is a stack (top is at the end)
Before we touch any code in the editor:

Would you rather:

keep using a simple list for the queue (with append and pop(0)), or
use collections.deque for the queue (with append and popleft)?

Pick one and we'll build the adventure around it.

im fairly hood with lists so lts use deque's collections

Love it — deque is a great choice 💪

So we'll have:

a queue of people using deque
each person has a stack of items using a simple Python list (top of stack = end of the list)

Conceptually, one element in the queue will look like this:

Python

```
["Alice", ["sword", "potion"]]
# name     stack (backpack)
# top of stack is "potion"
```

And the whole queue is a deque of those:

Python

```
from collections import deque

adventure_queue = deque([
    ["Alice", ["sword", "potion"]],
    ["Bob", ["shield"]],
])
```

Next step: I'd like you to set up a tiny version of this in the code editor:

Import deque.
Create an adventure_queue with 3 people, each with 1–3 items in their stack (backpack).
print(adventure_queue) to see the structure.

Go ahead and do this in the editor (Run or Submit when you're happy).

Tell me when it's run, and we'll start moving people through the queue and using their stacks 🎒

Python

```
from collections import deque

adventure_queue = deque([
    ["Alice", ["sword", "potion", "map"]],
    ["Bob", ["shield", "dagger"]],
    ["John", ["magical hat", "spell book"]],
])
print(adventure_queue)
```

Nice work setting that up, Ron 🎯

Your queue now looks like this (front on the left):

Text

```
[ Alice, Bob, John ]
```

And each has a stack (top item last):

Alice → ["sword", "potion", "map"] (top = "map")
Bob → ["shield", "dagger"] (top = "dagger")
John → ["magical hat", "spell book"] (top = "spell book")
Now let's use both structures together.

Step-by-step scenario:

The first adventurer in line enters the dungeon → we take from the front of the queue.
Inside, they quickly grab their top item from the backpack → we pop() from their stack.

So we will:

use queue behavior (FIFO) to choose who goes next
use stack behavior (LIFO) to choose which item they use first

Question for you (just reasoning, no code yet):

For each action below, say whether it should use the queue or the stack:

“Send the next adventurer into the dungeon.”
“Make that adventurer use their most recently added item.”

You can answer in plain words like:

1 → queue, 2 → stack

1 is queue, 2 is a stack

Exactly right ☑

1 → queue: “next adventurer” = first in line → FIFO
2 → stack: “most recently added item” = top of backpack → LIFO

Now let’s act this out in your code 🎮

Step-by-step, in the editor:

Take the first adventurer from the queue using popleft().
From their backpack (the inner list), take the top item using pop().
print both:
who entered
which item they used
Also print(adventure_queue) after, so we see the updated line.

Try to write this under your existing code in the editor (don’t paste it here), run it, and then tell me what you see happening in the output.

Python

```
from collections import deque

adventure_queue = deque([
    ["Alice", ["sword", "potion", "map"]],
    ["Bob", ["shield", "dagger"]],
    ["John", ["magical hat", "spell book"]],
])

print(adventure_queue)
removed_adventurer = adventure_queue.popleft()
item_removed = removed_adventurer[1].pop()

print(f'removed sdventurer was: {removed_adventurer[0]}')
print(f'the item used was: {item_removed}')
print(adventure_queue)
```

This is great, Ron 👏 You just combined both behaviors correctly.

Walkthrough of what your code did, step by step:

Initial queue (front on the left):

[Alice, Bob, John]

```
removed_adventurer = adventure_queue.popleft()
```

Takes Alice out (queue = FIFO), leaving:

[Bob, John]
item_removed = removed_adventurer[1].pop()

removed_adventurer[1] is Alice's backpack: ["sword", "potion", "map"]
pop() removes "map" → LIFO (top item)
Your prints match that:

removed adventurer was: Alice
the item used was: map
remaining queue: Bob, John

So: queue decided the person, stack decided the item ☑

Next small step:

Let's say Alice stays in the dungeon after using the item, so we don't put her back in line.

Now, imagine we want Bob (next in line) to also enter and use his top item.

In plain words (no code yet), which two data-structure operations will we need again?

You can answer like:

take person: ...? on the queue
take item: ...? on the backpack stack
popleft to remove bob from the queue and put him inside then pop to remove his top backack item

Perfectly reasoned 💯

From the queue: use popleft() → Bob is the next adventurer in line (FIFO).
From Bob's backpack stack: use pop() → he uses his most recent item (LIFO).

You've just described the whole "queue of people, each with a stack of items" pattern in your own words. That's the core idea of this lesson 🎯

Let's have you actually do this for Bob in code, right after Alice:

call popleft() again
then pop() on Bob's backpack
print who it was and which item they used
then print the final queue

Go ahead and update your code in the editor to process Bob too, run it, and tell me what the final queue looks like in the output.

Python

```
from collections import deque

adventure_queue = deque([
    ["Alice", ["sword", "potion", "map"]],
    ["Bob", ["shield", "dagger"]],
    ["John", ["magical hat", "spell book"]],
])

print(adventure_queue)
removed_adventurer = adventure_queue.popleft()
item_removed = removed_adventurer[1].pop()
```

```
print(f'removed sdventurer was: {removed_adventurer[0]}')
print(f'the item used was: {item_removed}')

removed_adventurer = adventure_queue.popleft()
item_removed = removed_adventurer[1].pop()

print(f'second removed sdventurer was: {removed_adventurer[0]}')
print(f'second persons item used was: {item_removed}')

print(adventure_queue)
Nice, Ron — your simulation is spot on ✨


Step-by-step, your program now does:


Start queue: [Alice, Bob, John]
popleft() → Alice leaves the line, then pop() → she uses map
Queue becomes: [Bob, John]
popleft() → Bob leaves the line, then pop() → he uses dagger
Final queue: [John] (still carrying ["magical hat", "spell book"])

So you've successfully used:


FIFO (queue) to control who acts next
LIFO (stack) to control which item gets used first
I want to check one key idea now:


Imagine we change the story:


"Each adventurer should use their oldest item first, not the newest."


In that case, would we still keep their backpack as a stack, or would a queue fit better for their items? 😮

Answer in words only (no code).

queue makes more logical sense

Exactly — queue fits better in that new rule ☑


Step-by-step:


If we want oldest item first, that's FIFO behavior.
FIFO = queue, so each backpack would act more like a little queue of items instead of a stack.

So:


Old story: "use most recent item" → backpack = stack
New story: "use oldest item" → backpack = queue

You're clearly seeing how LIFO vs FIFO changes behavior 🎯

Last thing I want to do in this lesson is a quick check using a small multiple□choice question.


Ready for a short one-question quiz on "stack vs queue" roles?

Yes, give me the quiz
```

What do you think? Pick the answer that feels right 👇

You are designing a small dungeon game.

There is a line of adventurers waiting at the gate.
Each adventurer has a pile of spells and must always cast the most recently learned spell first.
Which structure is the best fit for the line at the gate, and which is the best fit for the pile of spells?

Choose the option that matches (line, spells) in that order.

A.

(stack, stack)

B.

(stack, queue)

C.

(queue, stack)

D.

(queue, queue)

Nice job, Ron 🎉 You picked (queue, stack) — that's exactly right.

Quick recap:

The line at the gate is a queue → first adventurer to arrive should be the first to enter (FIFO).
The pile of spells is a stack → adventurer always casts the most recently learned spell first (LIFO).

You've clearly got the difference between when to use a stack vs a queue, and you applied it correctly in a mixed scenario. 🔁🎒

If you'd like, we can do:

one more tiny coding tweak to your dungeon simulation, or
start wrapping up this lesson and move on to the next one.

What do you prefer?

Wrap up the lesson

Here's your wrap-up, Ron 🎯

What you just mastered:

LIFO vs FIFO in action

Stack (LIFO): "last in, first out" → most recent thing is used first
Queue (FIFO): "first in, first out" → earliest thing is used first
You correctly chose:
browser history → stack
print jobs / adventurer line → queue
"oldest item first" → queue instead of stack
Choosing stack vs queue in real scenarios

"Next person to be served" → queue
"Most recent spell / page / action" → stack

You applied this cleanly in the dungeon quiz: (queue for line, stack for spells).
Integrating stacks and queues together

You built a deque of adventurers, each with a list as a stack of items:
popleft() decided who acts next (queue behavior).
pop() decided which item they used first (stack behavior).
That’s exactly the “queue of characters each carrying a stack” pattern.
You’re thinking about these structures the right way now: not as abstract things, but as tools you pick based on “who goes next?” and “which item goes next?” 💡

If you want extra practice later, good mini-challenges are:

Design your own tiny game world where people wait in a line and each has a pile/queue of tasks.
Try switching backpacks from stacks to queues and see how the behavior changes.
For now, you’re all set to move on 🚀
When you’re ready, hit Next Lesson in the Learn tab, and I’ll keep building from here with you.


```
from collections import deque

adventure_queue = deque([
    ["Alice", ["sword", "potion", "map"]],
    ["Bob", ["shield", "dagger"]],
    ["John", ["magical hat", "spell book"]],
])

print(adventure_queue)
removed_adventurer = adventure_queue.popleft()
item_removed = removed_adventurer[1].pop()

print(f'removed sdventurer was: {removed_adventurer[0]}')
print(f'the item used was: {item_removed}')

removed_adventurer = adventure_queue.popleft()
item_removed = removed_adventurer[1].pop()

print(f'second removed sdventurer was: {removed_adventurer[0]}')
print(f'second persons item used was: {item_removed}')

print(adventure_queue)
```